STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC - 26 - 0074	StratoRAM

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	8mm
Fluid density (ρ) =	1000kg/m^3
Dynamic Viscosity (μ) =	1x10^{-3}Pa.s
Required Flow range =	1-6 L/min
Nominal target flow for the live challenge =	3L/min

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (SMPS)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

CASE 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

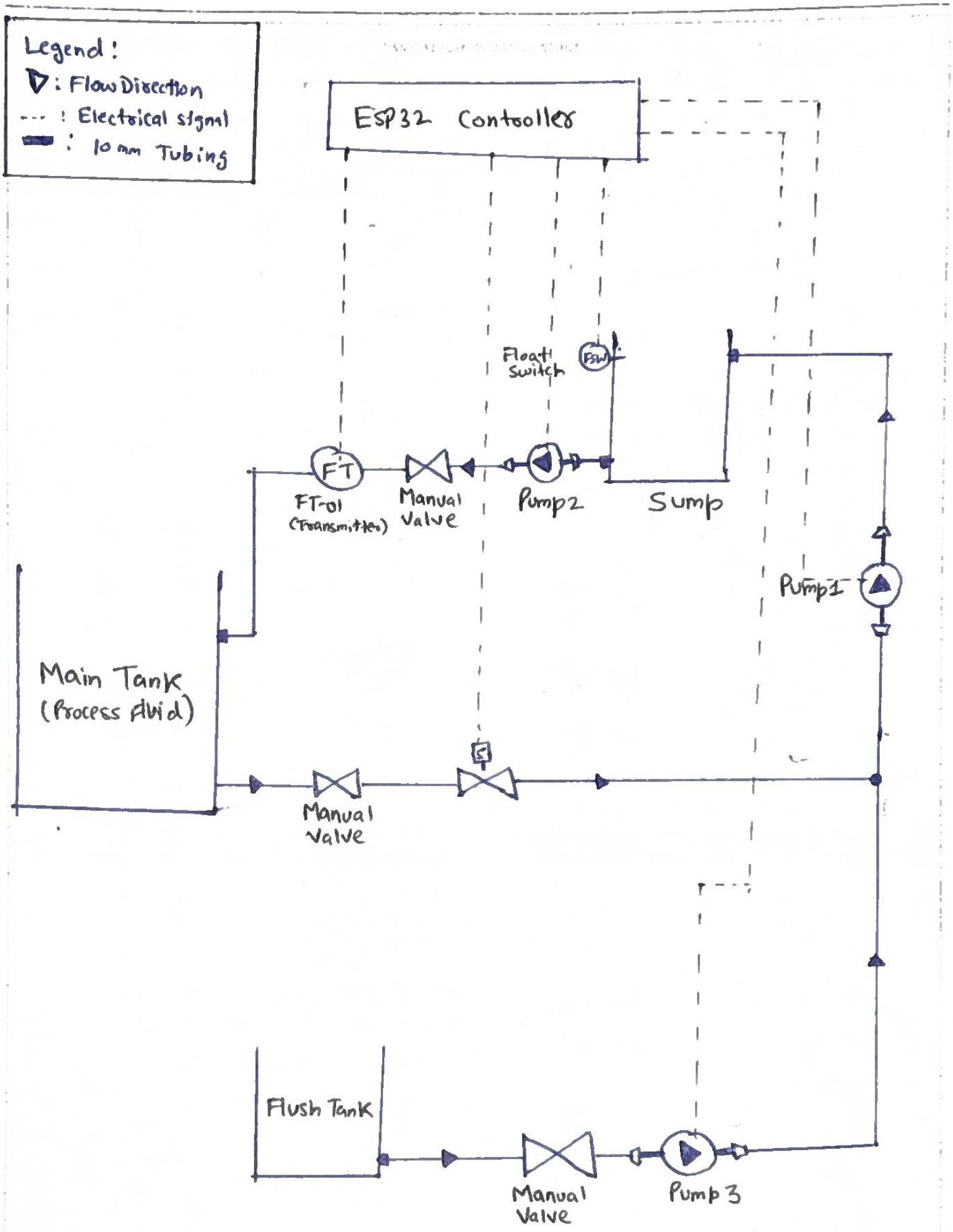
CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram


Legend :
: Flow Direction
: Electoical signal
: 10 mm Tubing
ESP32 Contoller
Float Switch
FSW
FT
FT-01
(Transmitter)
Manual Valve
Pump2
Sump
Pump1
Main Tank
(Process fluid)
Manual Valve
Flush Tank
Manual Valve
Pump 3

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Theoretical velocity calculations :

$v = \frac{Q}{A}$, v : velocity (m/s)
Q : flow rate
A : Area of cross section

diameter of pipe = 8mm = 0.008 m
radius (r) = 0.004m , $A = \pi r^2 = 5.026 \times 10^{-5} m^2$

At 1L/min :

$Q_1 = \frac{1 \times 10^{-3}}{60} = 1.67 \times 10^{-5}\ m^3/s$

$v_1 = \frac{Q_1}{A} = \frac{1.67 \times 10^{-5}}{5.026 \times 10^{-5}} = 0.33\ m/s$

At 3L/min :

$Q_2 = \frac{3 \times 10^{-3}}{60} = 5 \times 10^{-5}\ m^3/s$

$v_2 = \frac{Q_2}{A} = 0.99\ m/s$

At 6L/min :

$Q_3 = \frac{6 \times 10^{-3}}{60} = 10^{-4}\ m^3/s$

$v_3 = \frac{Q_3}{A} = 1.99\ m/s$

Actual operating behaviour :

In actual operation, the physical system behaves differently than the frictionless theoretical model. While 1L/min is easily achievable at a low PWM duty cycle, the system maxes out before reaching the higher ~~water~~ targets. Based on the system telemetry at 100% PWM : $Q_{max} \approx 2.32$ L/min $= 3.86 \times 10^{-5}\ m^3/s \Rightarrow v = 0.77$ m/s.

Possible Reasons :

→ Pump performance curve and operating point.
→ Power supply and Motor drive current limits.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE (L/min)
25%	0.775
50%	2.068
70%	2.292
100%	2.322

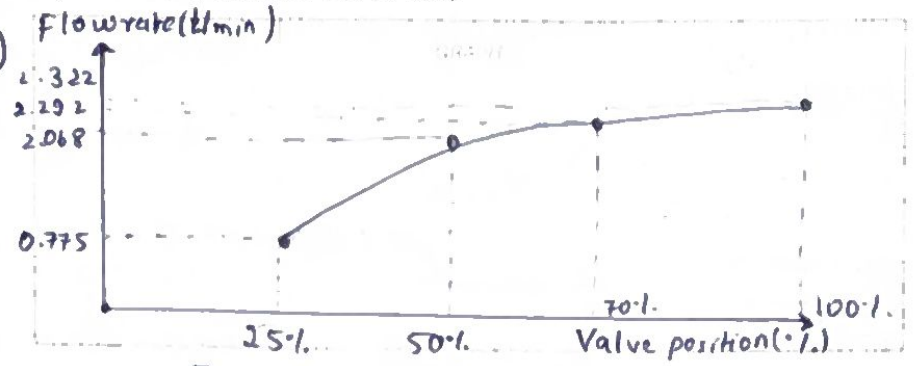


Fig. Flow rate vs Valve position graph.

Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 = 0.99083

a = 0.01018 $Lmin^{-1}Hz^{-1}$

b = −0.05975 L/min

Maximum calibration error = 0.0514 L/min (4.99%)

Average calibration error = 0.0408 L/min (2.63%)

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

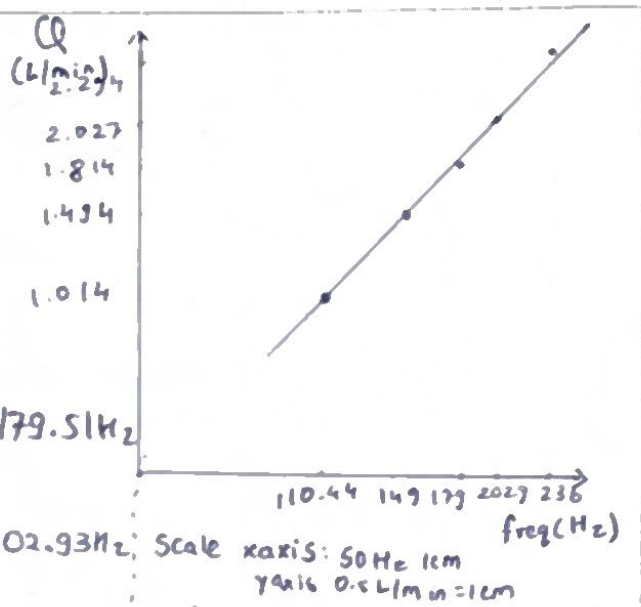
Data	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	4970	45	110.44	0.76	1.014
2	6712	45	149.16	1.12	1.494
3	8078	45	179.51	1.36	1.814
4	9132	45	202.93	1.52	2.027
5	10631	45	236.24	1.72	2.294

CALCULATIONS / OBSERVATIONS

Sample Calculations:-

→ we got direct pulses from flow transmitter signal and we can divide those by time which in our case is 45 seconds to get frequency f (in Hz).

$f = \frac{N}{t}$; $f_1 = \frac{4970}{45} = 110.44Hz$; $f_2 = \frac{6712}{45} = 149.16Hz$; $f_3 = \frac{8078}{45} = 179.51Hz$; $f_4 = \frac{9132}{45} = 202.93Hz$; $f_5 = \frac{10631}{45} = 236.24Hz$



Scale x axis: 50 Hz 1cm
y axis 0.5 L/min = 1cm

→ we collected volume manually ~~but~~ using measuring jug to get accurate readings and then we can get actual flow rate by dividing collected volume (V) with time

$Q = \frac{V}{t}$; $Q_1 = \frac{0.76}{45/60} = 1.014 L/min$; $Q_2 = \frac{1.12}{45/60} = 1.494 L/min$; $Q_3 = \frac{1.36}{45/60} = 1.814 L/min$; $Q_4 = \frac{1.52}{45/60} = 2.027 L/min$; $Q_5 = \frac{1.72}{45/60} = 2.294 L/min$

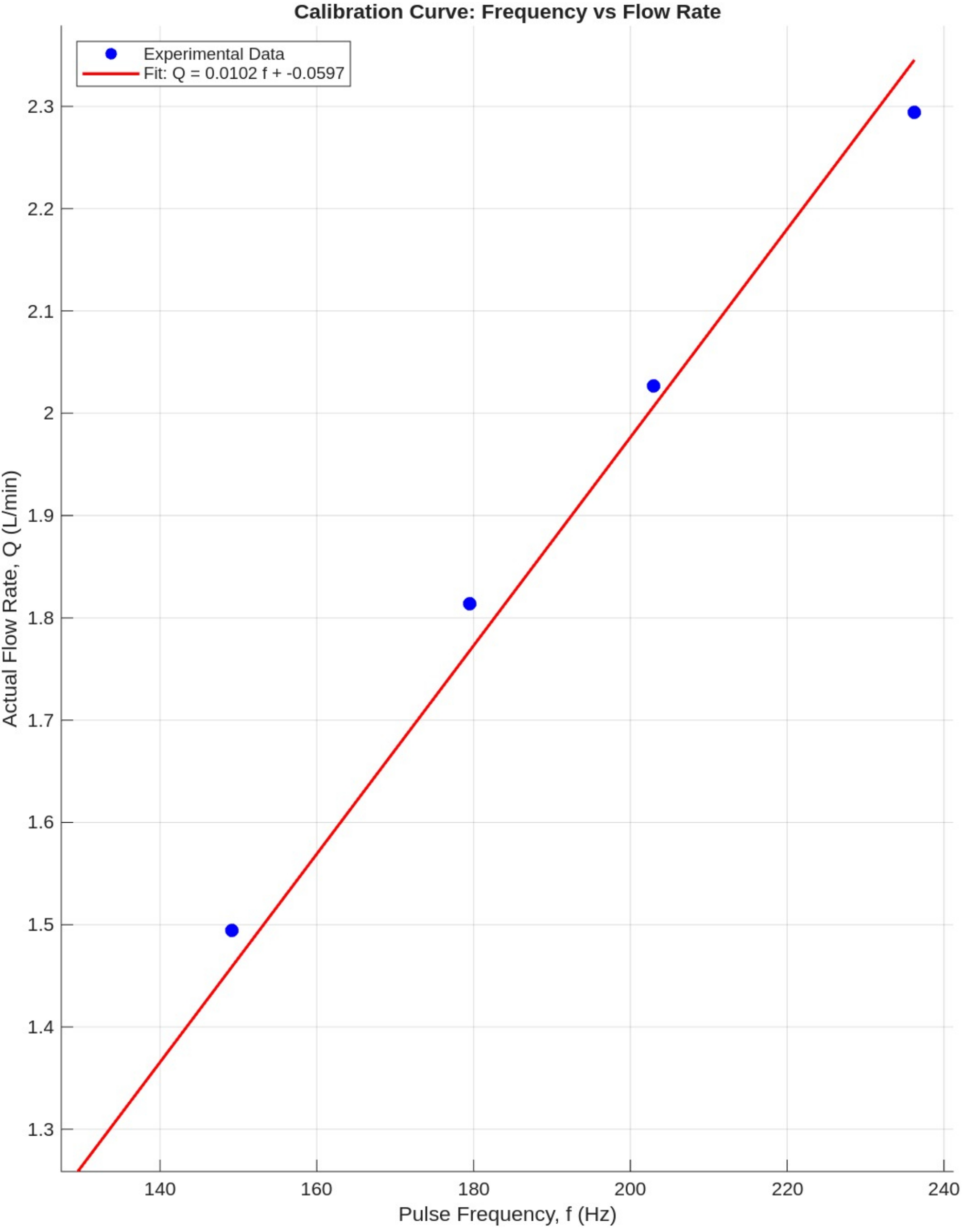
Observations:-

(1) Valve Characteristics:- Modulating the manual ball valve we realised that it demonstrated a non-linear flow control. Since ball valves are for ON/OFF and not precise throttling, change in flow rate from 100% to 70% is minimal but drops sharply beyond 50%.

(2) Sensor linearity:- YF-S401 flow transmitter demonstrated a highly linearly behaviour across operating range, validating R^2 of 0.99083. this strong correlation confirms proportionality b/w frequency and flow rate Q

(3) Sensor inertia:- Since b offset came out to be negative. It represents mechanical inertia, a minimum flow required to overcome friction and shows a minimum kinetic energy required by water

(4) Diminishing returns: The PWM vs flow rate, ~~case~~ we see that changing from 102 to 140 yielded 0.48 L/min increase but same 39 point increase 216 to 255 yielded only 0.26 L/min increase, proving that as velocity increases, system head loss through friction ($\propto$ sq of velocity) becomes dominant & leads to diminishing returns on power.


Calibration Curve: Frequency vs Flow Rate
Experimental Data
Fit: Q = 0.0102 f + -0.0597
Actual Flow Rate, Q (L/min)
2.3
2.2
2.1
2
1.9
1.8
1.7
1.6
1.5
1.4
1.3
140
160
180
200
220
240
Pulse Frequency, f (Hz)

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

Note: (There's a typo residence time is given by $T_r = V/Q$)

1. It represents the time duration a fluid particle spends while travelling through the section of tubing. Alternatively it also represents the time to flush the entire fluid out of the tubing.

2. $T_r = \frac{V}{Q}$ Q is constant $\Rightarrow T_r \propto V$, V doubles $\Rightarrow T_r$ doubles

~~If Q~~

3. $T_r = \frac{V}{Q}$ V is constant $\Rightarrow T_r \propto \frac{1}{Q}$, Q doubles $\Rightarrow T_r$ halves.

4. Residence time defines the system's physical transport lag. If a sudden change upstream (eg. a valve opens, or a new chemical is introduced), the residence time tells the engineer how much time it would take for that change to reach downstream ~~and~~ or register a change in downstream sensors.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

condition A → The system is in nominal operating condition with no faults

condition B → The Partially closed valve, introduces reduced flowrate, and less frequent Pulse generation.

condition C → Indicates a critical failure, This can be caused by blocked Pipe, empty fluid supply, jammed flow transmitter, or a dead pump motor.

condition D → The system is in nominal standby mode.

condition E → Indicates hydraulic instability, caused by air bubbles trapped in the lines, Pump cavitation or an intermittent electrical short in the pump/sensor wiring.

condition C is most difficult to diagnose as FT-01 Alone. because zero reading can mean anything like sensor jam, dead Pump, empty tanker.
To Properly rule out we would need → 1. Pump motor Power telemetry (dead motor)
2. A fluid level sensor in tank (Empty tank)
3. A Pressure transducer after the ~~tank~~ Pump. (for blockage)

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller.** It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

Final Working Code

```
#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>

// wifi credentials
const char* ssid = "ESP32_Water_Rig";
const char* password = "mypassword123";

WebServer server(80);
String messageLog = "System initialized. Waiting for Start.<br>";
int logCounter = 0;

// pin defs for the esp32 board
#define PIN_SOLENOID    27
#define PIN_FLOAT_SW    35
#define PIN_FLOW_SENSOR 14
#define PIN_FLUSH_BTN   34

// pump u2 pins
#define PIN_U2_LPWM     32
#define PIN_U2_RPWM     33
#define PIN_U2_REN      26
#define PIN_U2_LEN      13

// pump u3 pins
#define PIN_U3_LPWM     18
#define PIN_U3_REN      17
#define PIN_U3_LEN      16

// pump u4 pins
#define PIN_U4_RPWM     25
#define PIN_U4_LPWM     23
#define PIN_U4_REN      22
#define PIN_U4_LEN      21

// pwm settings
#define PWM_FREQ_HZ     1000
#define PWM_RES_BITS    8
const uint8_t DUTY_50_PERCENT = 128;
const uint8_t DUTY_100_PERCENT = 255;

// float states
const int ABOVE_LEVEL_STATE = LOW;
const int BELOW_LEVEL_STATE = HIGH;
int stableFloatState = -1;
int lastProcessedFloatState = -1;

// variables for the flow sensor
volatile uint32_t pulseCount = 0;
```

```
float flowRateLPM = 0.0;
float totalLitres = 0.0;
float currentFrequency = 0.0;
unsigned long lastFlowTime = 0;

// calib values we got from testing
const float CALIB_A = 0.01018;
const float CALIB_B = -0.05975;

// tracking stuff for safety
float flushVolumeLiters = 0.0;
int zeroFlowSeconds = 0;

// 5 sec log vars
uint32_t accumulatedPulses5s = 0;
unsigned long last5sLogTime = 0;

// button bounce fix
int currentBtnState = HIGH;
int lastBtnReading = HIGH;
unsigned long lastBtnDebounceTime = 0;
bool btnPressedDown = false;

// state machine modes
enum SystemMode { MODE_OFF_PRE_PHASE1, MODE_PHASE1, MODE_OFF_PRE_FLUSH, MODE_FLUSH
};
SystemMode currentMode = MODE_OFF_PRE_PHASE1;

int phaseStep = 0;
unsigned long sequenceTimer = 0;
bool systemErrorHalt = false;
String systemStatusText = "SYSTEM OFF";

// interrupt function to count pulses fast
void IRAM_ATTR pulseCounter() {
  pulseCount++;
}

// shut off all outputs
void turnEverythingOff() {
  digitalWrite(PIN_SOLENOID, LOW);
  ledcWrite(PIN_U2_LPWM, 0); ledcWrite(PIN_U2_RPWM, 0);
  ledcWrite(PIN_U3_LPWM, 0);
  ledcWrite(PIN_U4_RPWM, 0); ledcWrite(PIN_U4_LPWM, 0);
}

// adds logs to the web ui
void addLog(String msg) {
  logCounter++;
  String entry = "[" + String(millis() / 1000) + "s] " + msg + "<br>";
  messageLog = entry + messageLog;
  if (logCounter % 15 == 0) messageLog = entry;
  Serial.println(msg);
```

```
}

// html page
void handleRoot() {
  String html = "<!DOCTYPE html><html><head>";
  html += "<meta name='viewport' content='width=device-width, initial-scale=1.0'>";
  html += "<meta http-equiv='refresh' content='2'>";
  html += "<title>ESP32 Water Dashboard</title>";
  html += "<style>";
  html += "body { font-family: monospace; background: #121212; color: #00ff66;
padding: 15px; }";
  html += ".box { background: #1e1e1e; padding: 15px; border-radius: 8px; border:
1px solid #333; margin-bottom: 15px; }";
  html += ".val { font-size: 1.5em; font-weight: bold; color: #fff; }";
  html += ".warn { background: #ff0000; color: #fff; padding: 10px; text-align:
center; font-size: 1.2em; font-weight: bold; border-radius: 5px; animation: blinker
1s linear infinite; }";
  html += "@keyframes blinker { 50% { opacity: 0; } }";
  html += "</style></head><body>";

  html += "<h2>Water System Telemetry</h2>";

  // show red banner if halted
  if (systemErrorHalt) {
    html += "<div class='warn'>EMERGENCY HALT: " + systemStatusText + "</div><br>";
  }
  // show orange warning if tank is full in phase 1
  else if (currentMode == MODE_PHASE1 && stableFloatState == ABOVE_LEVEL_STATE) {
    html += "<div class='warn' style='background:#ffaa00; animation:none;'>OVERFLOW
WARNING: TANK FULL! (U3 STOPPED)</div><br>";
  }

  html += "<div class='box'>";
  html += "Current Mode: <span class='val' style='color:#00ffff;'>" +
systemStatusText + "</span><br><br>";

  // show live flush volume
  if (currentMode == MODE_FLUSH && phaseStep == 1 && !systemErrorHalt) {
    html += "Flush Progress: <span class='val' style='color:#ff00ff;'>" +
String(flushVolumeLiters, 3) + " / 2.0 L</span><br><br>";
  }

  // warning countdown for low flow
  if (zeroFlowSeconds > 0 && !systemErrorHalt) {
    int timeRemaining = 5 - zeroFlowSeconds;
    html += "Warning: <span class='val' style='color:#ff5500; animation: blinker 1s
linear infinite;'>LOW FLOW! Halt in " + String(timeRemaining) + "s...</span><br>
<br>";
  }

  html += "Volume Flow Rate (Q): <span class='val'>" + String(flowRateLPM, 3) + "
L/min</span><br><br>";
  html += "Frequency (f): <span class='val'>" + String(currentFrequency, 1) + "
```

```
Hz</span><br><br>";
  html += "Total Volume (V): <span class='val'>" + String(totalLitres, 3) + "
L</span>";
  html += "</div>";

  html += "<h3>System Log</h3>";
  html += "<div class='box'>" + messageLog + "</div>";

  html += "</body></html>";
  server.send(200, "text/html", html);
}

void setup() {
  Serial.begin(115200);
  // wifi setup
  WiFi.setSleep(false);
  WiFi.softAP(ssid, password);
  server.on("/", handleRoot);
  server.begin();
  // pin setups
  pinMode(PIN_FLUSH_BTN, INPUT_PULLUP);
  pinMode(PIN_FLOAT_SW, INPUT);
  pinMode(PIN_FLOW_SENSOR, INPUT);
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW_SENSOR), pulseCounter, FALLING);
  pinMode(PIN_SOLENOID, OUTPUT);
  pinMode(PIN_U2_REN, OUTPUT); pinMode(PIN_U2_LEN, OUTPUT);
  pinMode(PIN_U3_REN, OUTPUT); pinMode(PIN_U3_LEN, OUTPUT);
  pinMode(PIN_U4_REN, OUTPUT); pinMode(PIN_U4_LEN, OUTPUT);
  // start with everything off
  turnEverythingOff();
  digitalWrite(PIN_U2_REN, HIGH); digitalWrite(PIN_U2_LEN, HIGH);
  digitalWrite(PIN_U3_REN, HIGH); digitalWrite(PIN_U3_LEN, HIGH);
  digitalWrite(PIN_U4_REN, HIGH); digitalWrite(PIN_U4_LEN, HIGH);
  // attach pwm to pins
  ledcAttach(PIN_U2_LPWM, PWM_FREQ_HZ, PWM_RES_BITS); ledcWrite(PIN_U2_LPWM, 0);
  ledcAttach(PIN_U3_LPWM, PWM_FREQ_HZ, PWM_RES_BITS); ledcWrite(PIN_U3_LPWM, 0);
  ledcAttach(PIN_U4_RPWM, PWM_FREQ_HZ, PWM_RES_BITS); ledcWrite(PIN_U4_RPWM, 0);
  currentMode = MODE_OFF_PRE_PHASE1;
  systemStatusText = "SYSTEM OFF";
  addLog("Boot Complete: System OFF. Press Button to start Phase 1.");
}

void loop() {
  server.handleClient();
  // read flow sensor every 1 sec
  if (millis() - lastFlowTime >= 1000) {
    noInterrupts();
    uint32_t currentPulses = pulseCount;
    pulseCount = 0;
    interrupts();
    currentFrequency = currentPulses / 1.0;
    // calc lpm and prevent negative values from offset
    if (currentFrequency > 0) {
```

```
      flowRateLPM = (CALIB_A * currentFrequency) + CALIB_B;
      if (flowRateLPM < 0.0) flowRateLPM = 0.0;
    } else {
      flowRateLPM = 0.0;
    }
    // update totals
    float volumeThisSecond = (flowRateLPM / 60.0);
    totalLitres += volumeThisSecond;
    if (currentMode == MODE_FLUSH && phaseStep == 1) {
      flushVolumeLiters += volumeThisSecond;
    }
    // check if pumps are running but no flow
    bool isSystemPumping = (currentMode == MODE_PHASE1 && phaseStep == 2) ||
(currentMode == MODE_FLUSH && phaseStep == 1);
    if (isSystemPumping && !systemErrorHalt) {
      if (flowRateLPM <= 0.1) {
        zeroFlowSeconds++;
        if (zeroFlowSeconds >= 5) {
          systemErrorHalt = true;
          systemStatusText = "LOW FLOW ERROR";
          turnEverythingOff();
          addLog(">>> CRITICAL: Flow rate <= 0.1 L/min for 5 seconds. System
shutdown.");
        }
      } else {
        zeroFlowSeconds = 0; // reset if water is moving
      }
    } else {
      zeroFlowSeconds = 0; // keep at 0 if stopped
    }
    accumulatedPulses5s += currentPulses;
    lastFlowTime = millis();
  }
  // log data every 5 secs during phase 1
  if (millis() - last5sLogTime >= 5000) {
    if (currentMode == MODE_PHASE1 && phaseStep == 2 && !systemErrorHalt) {
      addLog("Data [5s] -> Pulses(N): " + String(accumulatedPulses5s) + " |
Time(t): 5s | Freq(f): " + String(accumulatedPulses5s / 5.0, 1) + " Hz");
    }
    accumulatedPulses5s = 0;
    last5sLogTime = millis();
  }
  // float switch logic
  int read1 = digitalRead(PIN_FLOAT_SW);
  delay(30); // small delay for bounce
  int read2 = digitalRead(PIN_FLOAT_SW);
  if (read1 == read2) {
    if (read1 != stableFloatState) {
      stableFloatState = read1;
      // instant stop if overflowing in flush mode
      if (currentMode == MODE_FLUSH && phaseStep == 1 && stableFloatState ==
ABOVE_LEVEL_STATE) {
        systemErrorHalt = true;
```

```
        systemStatusText = "OVERFLOW HALT";
        turnEverythingOff();
        addLog(">>> CRITICAL: Water exceeded float during Flush! System
shutdown.");
      }
    }
  }
  // cycle modes with push button
  int reading = digitalRead(PIN_FLUSH_BTN);
  if (reading != lastBtnReading) lastBtnDebounceTime = millis();
  if ((millis() - lastBtnDebounceTime) > 50) {
    if (reading != currentBtnState) {
      currentBtnState = reading;
      if (currentBtnState == LOW) btnPressedDown = true;
      if (currentBtnState == HIGH && btnPressedDown) {
        btnPressedDown = false;
        // if we are in an error state pushing the button resets it
        if (systemErrorHalt) {
          systemErrorHalt = false;
          currentMode = MODE_OFF_PRE_PHASE1;
          systemStatusText = "SYSTEM OFF";
          addLog("System reset from Error State -> OFF.");
        }
        else {
          // state machine transitions
          if (currentMode == MODE_OFF_PRE_PHASE1) {
            currentMode = MODE_PHASE1;
            phaseStep = 0;
            sequenceTimer = millis();
            systemStatusText = "PHASE 1 (Waiting 3s...)";
            turnEverythingOff();
            addLog("Button Pressed: PHASE 1 INIT.");
          }
          else if (currentMode == MODE_PHASE1) {
            currentMode = MODE_OFF_PRE_FLUSH;
            systemStatusText = "SYSTEM OFF";
            turnEverythingOff();
            addLog("Button Pressed: SYSTEM OFF.");
          }
          else if (currentMode == MODE_OFF_PRE_FLUSH) {
            currentMode = MODE_FLUSH;
            phaseStep = 0;
            sequenceTimer = millis();
            systemStatusText = "FLUSH (Waiting 5s...)";
            turnEverythingOff();
            addLog("Button Pressed: FLUSH MODE INIT.");
          }
          else if (currentMode == MODE_FLUSH) {
            currentMode = MODE_OFF_PRE_PHASE1;
            systemStatusText = "SYSTEM OFF";
            turnEverythingOff();
            addLog("Button Pressed: SYSTEM OFF.");
          }
```

```
      }
    }
  }
}
lastBtnReading = reading;
// run the actual modes
if (!systemErrorHalt) {
  // normal phase 1 logic
  if (currentMode == MODE_PHASE1) {
    if (phaseStep == 0) {
      // wait 3s then open valve
      if (millis() - sequenceTimer >= 3000) {
        digitalWrite(PIN_SOLENOID, HIGH);
        systemStatusText = "PHASE 1 (Solenoid Open, Wait 5s...)";
        addLog("Solenoid OPEN. Waiting 5s...");
        sequenceTimer = millis();
        phaseStep = 1;
      }
    }
    else if (phaseStep == 1) {
      // wait 5s then turn on u4
      if (millis() - sequenceTimer >= 5000) {
        ledcWrite(PIN_U4_RPWM, DUTY_100_PERCENT);
        systemStatusText = "PHASE 1 (Running)";
        addLog("U4 Continuous ON (100%). Float logic engaged.");
        lastProcessedFloatState = -1;
        zeroFlowSeconds = 0; // reset the safety timer
        phaseStep = 2;
      }
    }
    else if (phaseStep == 2) {
      // control u3 based on float switch immediately
      if (stableFloatState != lastProcessedFloatState) {
        if (stableFloatState == ABOVE_LEVEL_STATE) {
          ledcWrite(PIN_U3_LPWM, 0);
          systemStatusText = "PHASE 1 (OVERFLOW - U3 OFF)";
          addLog("Float HIGH -> U3 stopped instantly. OVERFLOW TRIGGERED.");
        } else {
          ledcWrite(PIN_U3_LPWM, DUTY_50_PERCENT);
          systemStatusText = "PHASE 1 (Running)";
          addLog("Float LOW -> U3 ON (50%).");
        }
        lastProcessedFloatState = stableFloatState;
      }
    }
  }
  // flush sequence logic
  else if (currentMode == MODE_FLUSH) {
    if (phaseStep == 0) {
      // wait 5 sec then fire all pumps
      if (millis() - sequenceTimer >= 5000) {
        ledcWrite(PIN_U2_LPWM, DUTY_100_PERCENT);
        ledcWrite(PIN_U3_LPWM, DUTY_100_PERCENT);
```

```
          ledcWrite(PIN_U4_RPWM, DUTY_100_PERCENT);
          systemStatusText = "FLUSHING (Target: 2 Liters)";
          addLog("5s elapsed. U2, U3, U4 running at 100%. Waiting for 2.0L.");
          flushVolumeLiters = 0.0; // reset flush counter
          zeroFlowSeconds = 0;
          phaseStep = 1;
        }
      }
      else if (phaseStep == 1) {
        // turn off after hitting 2 liters
        if (flushVolumeLiters >= 2.0) {
          currentMode = MODE_OFF_PRE_PHASE1;
          systemStatusText = "SYSTEM OFF";
          turnEverythingOff();
          addLog("2.0 Liters flushed successfully. System OFF.");
        }
      }
    }
  }
}
```

Solenoid Valve

```
#include <Arduino.h>
// quick script to test just the solenoid valve
// wired to pin 27 on your pcb
#define PIN_SOLENOID 27
void setup() {
  Serial.begin(115200);
  delay(1000);
  // set pin as output
  pinMode(PIN_SOLENOID, OUTPUT);
  // ensure it starts shut
  digitalWrite(PIN_SOLENOID, LOW);
  Serial.println("starting solenoid test...");
}
void loop() {
  // turn it on
  Serial.println("solenoid OPEN (listen for the click)");
  digitalWrite(PIN_SOLENOID, HIGH);
  delay(3000); // leave open for 3 seconds

  // turn it off
  Serial.println("solenoid CLOSED");
  digitalWrite(PIN_SOLENOID, LOW);
  delay(3000); // keep it off for 3 seconds before repeating
}
```

Pumps

```
#include <Arduino.h>
// simple script to test a single pump and driver board
// testing pump u4 on pins 25, 23, 22, 21
#define PIN_U4_RPWM 25
#define PIN_U4_LPWM 23
#define PIN_U4_REN  22
#define PIN_U4_LEN  21
void setup() {
  Serial.begin(115200);
  delay(1000);
  // setup driver enable pins
  pinMode(PIN_U4_REN, OUTPUT);
  pinMode(PIN_U4_LEN, OUTPUT);
  // turn both enables on
  digitalWrite(PIN_U4_REN, HIGH);
  digitalWrite(PIN_U4_LEN, HIGH);
  // attach pwm to the esp32 hardware channels
  ledcAttach(PIN_U4_RPWM, 1000, 8);
  ledcWrite(PIN_U4_RPWM, 0);
  Serial.println("pump u4 test started");
}

void loop() {
  Serial.println("ramping up to 50% power...");
  ledcWrite(PIN_U4_RPWM, 128);
  delay(3000); // run for 3 seconds
  Serial.println("ramping up to 100% power...");
  ledcWrite(PIN_U4_RPWM, 255);
  delay(3000);
  Serial.println("stopping pump...");
  ledcWrite(PIN_U4_RPWM, 0);
  delay(3000); // stay off for 3 seconds before repeating
}
```

Float Switch

```
#include <Arduino.h>

// simple code to check if float switch is working
// hooked up to gpio 35
#define PIN_FLOAT_SW 35
void setup() {
  Serial.begin(115200);
  delay(1000);
  // set pin as input
  pinMode(PIN_FLOAT_SW, INPUT);
  Serial.println("float switch monitor started");
  Serial.println("move the float up and down to see state changes");
}

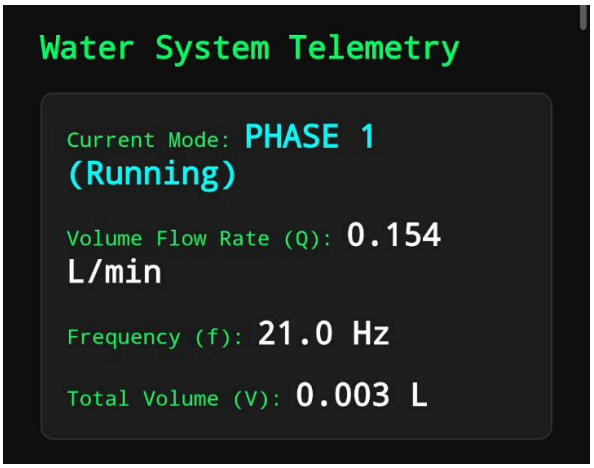
void loop() {
  // read the current state of the switch
  int floatState = digitalRead(PIN_FLOAT_SW);
  // print the output to serial monitor
  if (floatState == LOW) {
    Serial.println("state: LOW (float is UP / tank is full)");
  } else {
    Serial.println("state: HIGH (float is DOWN / tank is empty)");
  }
  // wait half a second so it doesnt spam the console too fast
  delay(500);
}
```

Flow transmitter

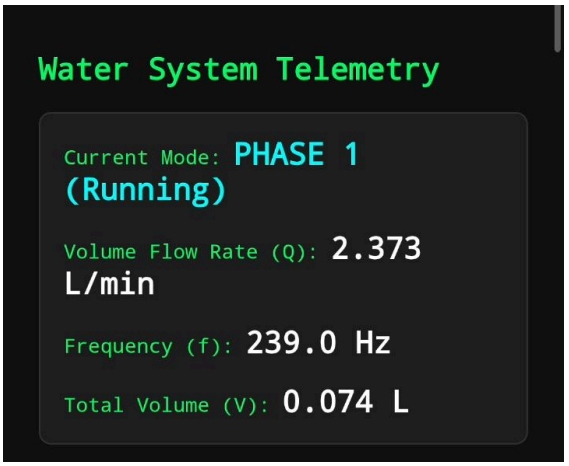
```
#include <Arduino.h>

// simple code to test if the flow transmitter is reading
// connected to pin 14
#define PIN_FLOW_SENSOR 14
// vars to keep track of pulses
volatile uint32_t pulseCount = 0;
uint32_t totalPulses = 0;
unsigned long lastTime = 0;
// fast interrupt to catch every single pulse
void IRAM_ATTR countPulses() {
  pulseCount++;
}
void setup() {
  Serial.begin(115200);
  delay(1000);
  // pullup resistor just to be safe with the hall effect chip
  pinMode(PIN_FLOW_SENSOR, INPUT_PULLUP);
  // trigger on the falling edge of the magnet pass
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW_SENSOR), countPulses, FALLING);
  Serial.println("flow sensor test started");
  Serial.println("blow air into the tube to see the numbers go up");
}
void loop() {
  // check exactly every 1 second
  if (millis() - lastTime >= 1000) {
    // pause interrupts super fast to copy the number safely
    noInterrupts();
    uint32_t currentPulses = pulseCount;
    pulseCount = 0;
    interrupts();
    totalPulses += currentPulses;
    lastTime = millis();
    // print the results to serial monitor
    Serial.print("pulses this sec: ");
    Serial.print(currentPulses);
    Serial.print(" hz  |  total pulses: ");
    Serial.println(totalPulses);
  }
}
```

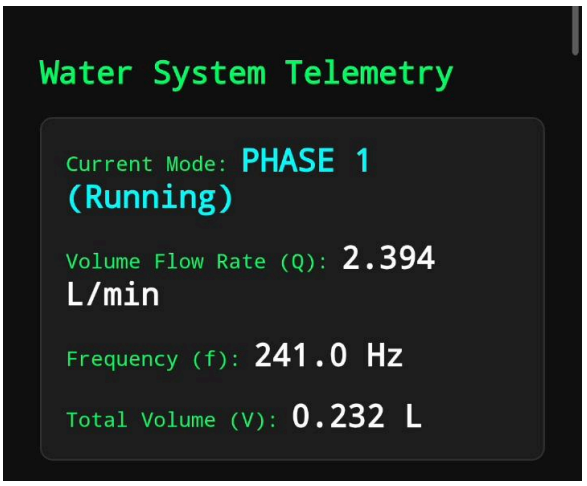
Flow v/s Time Trend



Starting



Running



Steady State